# LAPORAN TUGAS
# DEEP LEARNING

Klasifikasi Data Bla Bla Bla dengan TensorFlow Keras – Machine Learning Pertemuan 11

**NAMA : Muhammad Irfan Nuha**
**NIM : 434241079**
**Kelas : TI-C5**

Tugas Individu

# 1. Pendahuluan

**Deep learning** adalah cabang machine learning yang memakai **jaringan saraf tiruan** (*artificial neural network*) berlapis-lapis. Setiap lapisan terdiri dari neuron yang menghitung jumlah berbobot dari masukannya lalu melewatkannya ke fungsi aktivasi. Bobot-bobot ini dipelajari otomatis dari data selama proses *training*, sehingga jaringan mampu menangkap pola yang rumit.

Tugas ini melengkapi program pada modul untuk membangun model klasifikasi dengan **TensorFlow Keras** pada dataset **Bla Bla Bla** dari pertemuan sebelumnya. Dataset berisi 2.308 data pasien dengan kolom: kode umur (A), umur dalam tahun (UMUR_TAHUN), jenis kelamin (B), sebelas gejala berbentuk biner 0/1 (C sampai M), dan kelas diagnosis **N** (0 = negatif, 1 = positif).

# 2. Konsep Jaringan Saraf Tiruan

Komponen jaringan yang dipakai pada modul:

- **Dense**: lapisan yang setiap neuronnya terhubung ke semua neuron lapisan sebelumnya.

- **ReLU** (*Rectified Linear Unit*): aktivasi pada lapisan tersembunyi, mengubah nilai negatif menjadi 0 sehingga jaringan bisa mempelajari hubungan tidak linear.

- **Sigmoid**: aktivasi pada lapisan output, menghasilkan probabilitas 0 sampai 1 untuk kelas positif.

- **Binary crossentropy**: fungsi loss untuk klasifikasi dua kelas; makin kecil makin baik.

- **Adam**: optimizer yang memperbarui bobot secara adaptif berdasarkan gradien loss.

- **Epoch**: satu kali putaran seluruh data training; model dilatih 100 epoch dengan 20% data training disisihkan sebagai data validasi (*validation_split*) untuk memantau overfitting.

# 3. Import Library, Load Dataset, dan Pemisahan Fitur

py deep_learning_434241079.py

```python
# 1. Import Library
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.metrics import confusion_matrix, classification_report
import tensorflow as tf

tf.keras.utils.set_random_seed(42)              # agar hasil training bisa diulang

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
OUT_DIR = os.path.join(BASE_DIR, "output")
os.makedirs(OUT_DIR, exist_ok=True)


def judul(teks):
    print("\n" + "=" * 62)
    print(f"  {teks}")
    print("=" * 62)


def simpan_dan_tampilkan(nama_file):
    plt.savefig(os.path.join(OUT_DIR, nama_file), dpi=110, bbox_inches="tight")
    plt.show()
    plt.close()
```

*Kode 1. Import library (langkah 1)*

py deep_learning_434241079.py

```python
# 2. Load dataset
judul("2. LOAD DATASET")
df = pd.read_excel(os.path.join(BASE_DIR, "BlaBla.xlsx"))
print(f"Ukuran data : {df.shape[0]} baris x {df.shape[1]} kolom")
print(df.head().to_string(index=False))

# 3. Pisahkan fitur dan label
judul("3. PISAHKAN FITUR DAN LABEL")
FITUR = ["Umur"] + list("BCDEFGHIJKLM")     # variabel input : Umur, B sampai M
LABEL = "N"                                  # variabel output: N
print(f"   Input  ({len(FITUR)} fitur) : {FITUR}")
print(f"   Output           : {LABEL}  ->  {df[LABEL].value_counts().sort_index().to_dict()}")
```

*Kode 2. Load dataset dan pemisahan fitur-label (langkah 2-3)*

Langkah 1 sampai 3 mengimpor library sesuai modul, membaca *BlaBla.xlsx* dengan *pd.read_excel()*, lalu memisahkan variabel. Sesuai modul, **input** adalah Umur serta kolom B sampai M (13 fitur), dan **output** adalah kolom N. Distribusi kelas tidak seimbang: 1.629 negatif dan 679 positif.

# 4. Encoding Umur

py deep_learning_434241079.py

```
59  # 4. Lakukan Encoding -> Umur
60  judul("4. ENCODING UMUR")
61  umur_tahun = pd.to_numeric(df["UMUR_TAHUN"], errors="coerce")   # teks salah ketik -> NaN
62
63
64  def kode_umur(u):
65      if pd.isna(u):
66          return np.nan
67      if u <= 20:
68          return 1          # <= 20
69      if u <= 30:
70          return 2          # 21 - 30
71      if u <= 40:
72          return 3          # 31 - 40
73      if u <= 50:
74          return 4          # 41 - 50
75      return 5              # > 50
76
77
78  df["Umur"] = umur_tahun.apply(kode_umur)
79  print("   Aturan: 1 = <=20 | 2 = 21-30 | 3 = 31-40 | 4 = 41-50 | 5 = >50")
80  print(f"   Sebaran kode Umur: {df['Umur'].value_counts().sort_index().to_dict()}")
81  sama = (df["Umur"].dropna() == df.loc[df["Umur"].notna(), "A"]).mean() * 100
82  print(f"   Kecocokan dengan kolom A pada data asli: {sama:.1f}%")
```

*Kode 3. Encoding umur (langkah 4)*

Windows PowerShell

```
PS D:\KULIAH\...\Pertemuan 11 - Deep Learning> python deep_learning_434241079.py

============================================================
  2. LOAD DATASET
============================================================
Ukuran data : 2308 baris x 15 kolom
 A UMUR_TAHUN  B  C  D  E  F  G  H  I  J  K  L  M  N
 1         17  0  1  0  0  0  0  0  1  0  0  0  1  0
 5         70  0  0  0  0  0  0  0  1  1  1  0  1  1
 3         39  0  0  0  0  0  1  0  0  0  0  0  1  0
 5         63  0  0  0  0  0  0  0  0  1  0  0  1  0
 3         40  0  0  0  0  0  1  0  0  0  1  0  1  0

============================================================
  3. PISAHKAN FITUR DAN LABEL
============================================================
   Input  (13 fitur) : ['Umur', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M']
   Output           : N  ->  {0: 1629, 1: 679}

============================================================
  4. ENCODING UMUR
============================================================
   Aturan: 1 = <=20 | 2 = 21-30 | 3 = 31-40 | 4 = 41-50 | 5 = >50
   Sebaran kode Umur: {1.0: 785, 2.0: 566, 3.0: 451, 4.0: 294, 5.0: 211}
   Kecocokan dengan kolom A pada data asli: 100.0%
```

*Output load dataset, pemisahan fitur, dan encoding umur*

Langkah 4 meng-*encoding* kolom UMUR_TAHUN menjadi kode 1 sampai 5 sesuai aturan modul (1 = sampai 20 tahun, 2 = 21-30, 3 = 31-40, 4 = 41-50, 5 = di atas 50) melalui fungsi *kode_umur()*. Sebelum itu, UMUR_TAHUN diubah ke angka dengan *pd.to_numeric(errors="coerce")* karena ternyata ada isian yang bukan angka. Hasil encoding kami cocok **100%** dengan kolom A pada data asli, yang membuktikan aturan encoding diterapkan dengan benar.

# 5. Preprocessing dan Split Data

py deep_learning_434241079.py

```
84  # 5. Lakukan Preprocessing Data
85  judul("5. PREPROCESSING DATA")
86  # a. Cek Missing Value
87  print(">> a. Missing value:")
88  salah = df.loc[umur_tahun.isna(), "UMUR_TAHUN"].tolist()
89  print(f"     UMUR_TAHUN bukan angka : {len(salah)} baris {salah}")
90  print(f"     Missing pada fitur     : {df[FITUR].isna().sum()[df[FITUR].isna().sum() > 0].to_dict()}")
91  df["Umur"] = df["Umur"].fillna(df["A"])      # isi dari kode umur kolom A (data asli)
92  print(f"     -> diisi dari kolom A, sisa missing = {int(df[FITUR + [LABEL]].isna().sum().sum())}")
93
94  # b. Outlier
95  print(">> b. Outlier:")
96  q1, q3 = umur_tahun.quantile(0.25), umur_tahun.quantile(0.75)
97  iqr = q3 - q1
98  bawah, atas = q1 - 1.5 * iqr, q3 + 1.5 * iqr
99  n_out = int(((umur_tahun < bawah) | (umur_tahun > atas)).sum())
100 print(f"     UMUR_TAHUN batas IQR [{bawah:.1f}, {atas:.1f}] -> outlier {n_out} baris")
101 print(f"     UMUR_TAHUN bernilai 0 tahun: {int((umur_tahun == 0).sum())} baris")
102 biner = all(set(df[c].unique()) <= {0, 1} for c in list("BCDEFGHIJKLM") + [LABEL])
103 print(f"     Kolom B-M dan N hanya berisi 0/1: {biner} -> tidak mungkin ada outlier")
104 print("     -> tidak dibuang: setelah encoding semua umur masuk kode 1-5")
105
106 X = df[FITUR].astype(float)
107 y = df[LABEL]
```

*Kode 4. Preprocessing: missing value dan outlier (langkah 5)*

Langkah 5 melakukan preprocessing:

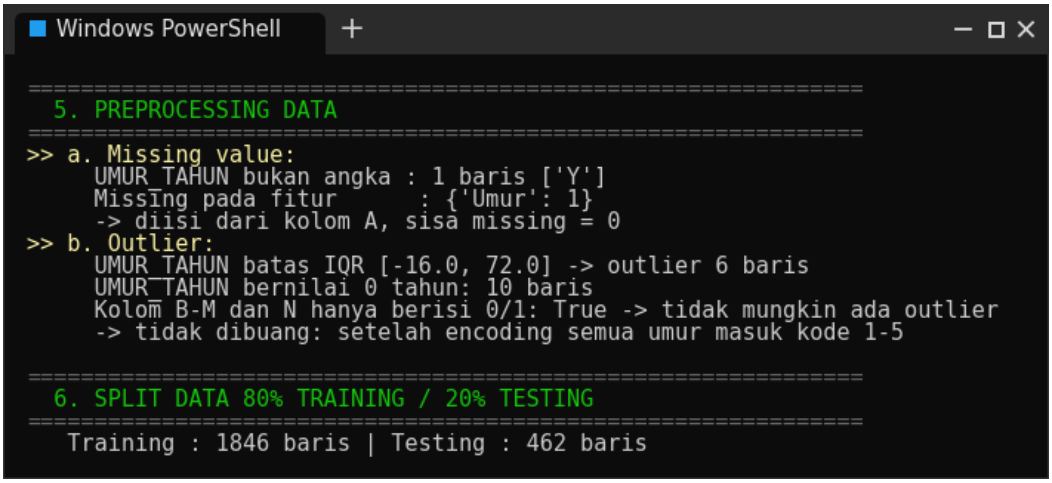
- **Missing value**: ditemukan satu nilai UMUR_TAHUN berisi huruf "**Y**" (salah input, kemungkinan tertukar dengan kolom gejala Y/N). Nilai ini menjadi kosong setelah konversi, lalu kami isi dengan kode umur dari kolom A pada baris yang sama, karena kolom A memang menyimpan kode umur pasien tersebut.

- **Outlier**: metode IQR pada UMUR_TAHUN menemukan 6 data di atas 72 tahun, dan ada 10 pasien tercatat berumur 0 tahun. Data ini tidak dibuang karena setelah di-encoding semuanya tetap masuk kode yang sah (0 tahun masuk kode 1, di atas 72 tahun masuk kode 5). Kolom B sampai M dan N hanya berisi 0 dan 1, sehingga tidak mungkin memiliki outlier.

py deep_learning_434241079.py

```
109 # 6. Split data (training 80%, testing 20%)
110 judul("6. SPLIT DATA 80% TRAINING / 20% TESTING")
111 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
112 scaler = StandardScaler()                    # standarisasi agar skala fitur seragam
113 X_train = scaler.fit_transform(X_train)
114 X_test = scaler.transform(X_test)
115 print(f"   Training : {len(X_train)} baris | Testing : {len(X_test)} baris")
116
117 # 7. Bangun model klasifikasi
118 judul("7. BANGUN MODEL")
119 model = Sequential([
120     Dense(16, activation='relu', input_shape=(X.shape[1],)),
121     Dense(8, activation='relu'),
122     Dense(1, activation='sigmoid')  # Output binary
123 ])
124 model.summary()
```

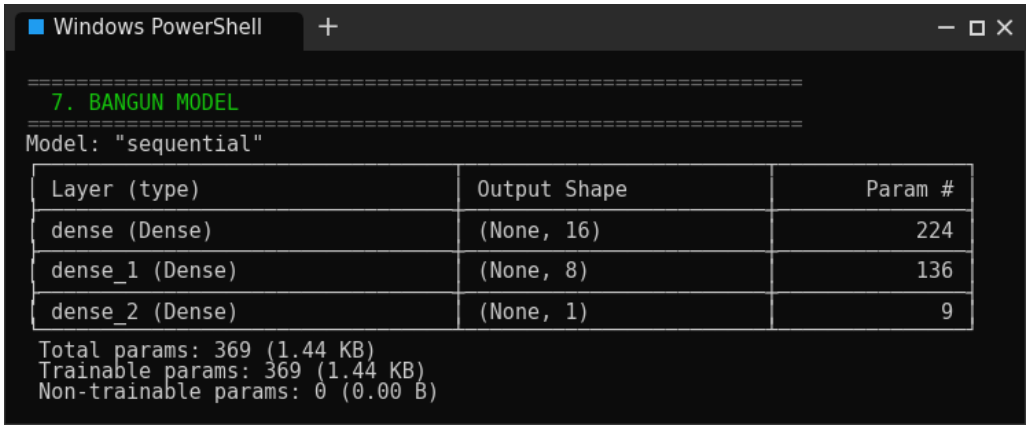
*Kode 5. Split data, standarisasi, dan pembangunan model (langkah 6-7)*




*Output preprocessing dan split data*

Langkah 6 membagi data menjadi **80% training (1.846 data)** dan **20% testing (462 data)** dengan *stratify* agar proporsi kelas positif dan negatif sama di kedua bagian. Fitur kemudian distandarisasi dengan **StandardScaler** (sudah diimpor di modul): rata-rata 0 dan simpangan baku 1, dihitung hanya dari data training agar tidak terjadi kebocoran informasi dari data testing.

# 6. Arsitektur Model




*Output model.summary()*

Langkah 7 membangun model *Sequential* persis seperti modul: lapisan tersembunyi pertama **16 neuron ReLU**, lapisan kedua **8 neuron ReLU**, dan lapisan output **1 neuron sigmoid**. Ringkasan model menunjukkan total **369 parameter** yang dipelajari:

- Dense 16: 13 input × 16 neuron + 16 bias = 224 parameter.
- Dense 8: 16 × 8 + 8 bias = 136 parameter.
- Dense 1: 8 × 1 + 1 bias = 9 parameter.

# 7. Compile dan Training

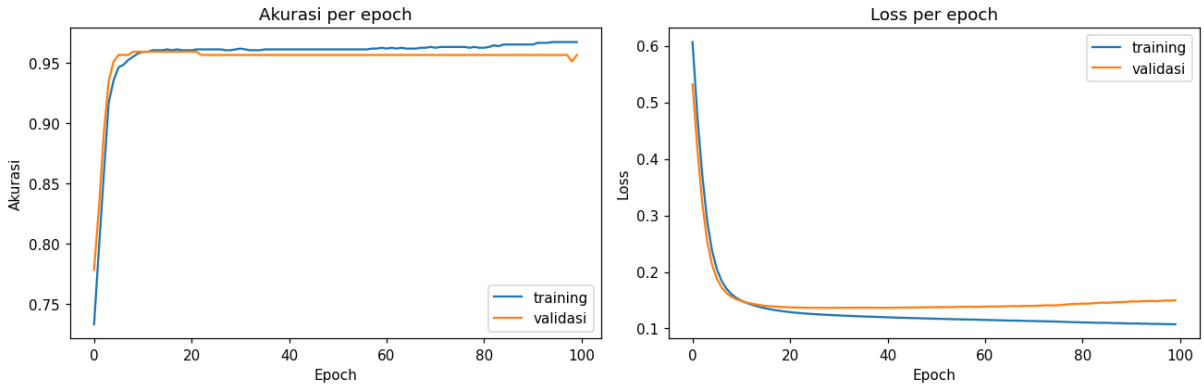
```python
# 8. Compile dan train
judul("8. COMPILE DAN TRAIN (100 epoch)")
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
history = model.fit(X_train, y_train, epochs=100, validation_split=0.2, verbose=1)

plt.figure(figsize=(12, 4))
for i, (m, nama) in enumerate([("accuracy", "Akurasi"), ("loss", "Loss")], 1):
    plt.subplot(1, 2, i)
    plt.plot(history.history[m], label="training")
    plt.plot(history.history["val_" + m], label="validasi")
    plt.xlabel("Epoch")
    plt.ylabel(nama)
    plt.title(f"{nama} per epoch")
    plt.legend()
plt.tight_layout()
simpan_dan_tampilkan("dl_1_history.png")
```

*Kode 6. Compile, training, dan grafik riwayat training (langkah 8)*

```
Windows PowerShell
============================================================
  8. COMPILE DAN TRAIN (100 epoch)
============================================================
Epoch 1/100
47/47 ━━━━━━━━━━━━━━━━━━━━ 2s 12ms/step - accuracy: 0.7331 - loss: 0.6067 - val_accuracy: 0.7784 - val_loss: 0.5319
Epoch 2/100
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7961 - loss: 0.4744 - val_accuracy: 0.8297 - val_loss: 0.4163
Epoch 3/100
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8577 - loss: 0.3712 - val_accuracy: 0.8919 - val_loss: 0.3214
  ... (epoch 4 sampai 97) ...
Epoch 98/100
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9675 - loss: 0.1077 - val_accuracy: 0.9568 - val_loss: 0.1494
Epoch 99/100
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9675 - loss: 0.1076 - val_accuracy: 0.9514 - val_loss: 0.1493
Epoch 100/100
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9675 - loss: 0.1074 - val_accuracy: 0.9568 - val_loss: 0.1499
```

*Output training (epoch awal dan akhir)*




*Akurasi dan loss per epoch (training vs validasi)*

Langkah 8 meng-*compile* model dengan optimizer **adam**, loss **binary_crossentropy**, dan metrik **accuracy**, lalu melatihnya 100 epoch. Pada epoch pertama akurasi validasi baru sekitar 78%, kemudian naik cepat dan stabil. Di epoch terakhir akurasi training **96,75%** dan akurasi validasi **95,68%**, dengan loss validasi 0,1499. Grafik akurasi memperlihatkan kurva training dan validasi bergerak berdampingan dengan jarak kecil. Pada grafik loss, loss validasi turun tajam sampai sekitar epoch 25, lalu mendatar dan sedikit naik menjelang epoch 100 sementara loss training terus turun. Ini tanda awal **overfitting ringan**: setelah sekitar 25 epoch model mulai menghafal data training. Dampaknya kecil karena akurasi validasi tetap stabil di kisaran 95-96%, tetapi jumlah epoch sebenarnya bisa dikurangi atau dihentikan otomatis dengan *EarlyStopping*.

# 8. Evaluasi, Confusion Matrix, dan Laporan Klasifikasi

py deep_learning_434241079.py

```python
# 9. Evaluasi
judul("9. EVALUASI")
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print(f"\nAkurasi di data uji: {acc * 100:.2f}%")
print(f"Loss di data uji   : {loss:.4f}")

# 10. Confusion matrix
judul("10. CONFUSION MATRIX")
y_pred = (model.predict(X_test, verbose=0) > 0.5).astype(int).ravel()   # probabilitas -> kelas 0/1
cm = confusion_matrix(y_test, y_pred)
print(cm)

# 11. Tampilkan confusion matrix
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=[0, 1], yticklabels=[0, 1])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
simpan_dan_tampilkan("dl_2_confusion_matrix.png")

# 12. Laporan klasifikasi
judul("12. LAPORAN KLASIFIKASI")
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

*Kode 7. Evaluasi, confusion matrix, dan classification report (langkah 9-12)*

Windows PowerShell

```
============================================================
  9. EVALUASI
============================================================

Akurasi di data uji: 95.45%
Loss di data uji   : 0.1625

============================================================
  10. CONFUSION MATRIX
============================================================
[[316  10]
 [ 11 125]]

============================================================
  12. LAPORAN KLASIFIKASI
============================================================

Classification Report:
              precision    recall  f1-score   support

           0       0.97      0.97      0.97       326
           1       0.93      0.92      0.92       136

    accuracy                           0.95       462
   macro avg       0.95      0.94      0.95       462
weighted avg       0.95      0.95      0.95       462
```

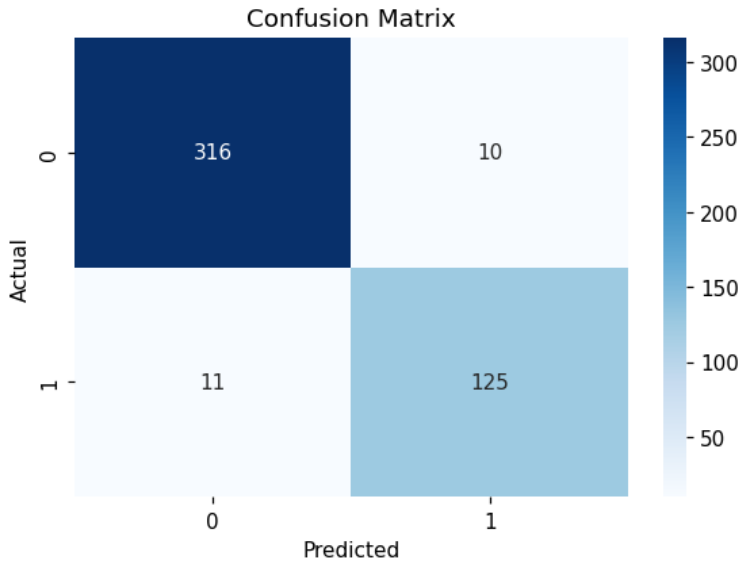
*Output evaluasi dan classification report*

Pada data uji yang belum pernah dilihat model, akurasinya **95,45%** dengan loss **0,1625**. Untuk mendapatkan kelas prediksi (*y_pred*), probabilitas keluaran sigmoid diubah menjadi 1 jika lebih dari 0,5 dan 0 jika sebaliknya; langkah ini kami tambahkan karena *y_pred* dipakai pada modul tetapi belum didefinisikan.

Confusion matrix dibaca sebagai berikut (kelas positif = 1):

- **TN = 316**: pasien negatif yang tertebak negatif.
- **FP = 10**: pasien negatif yang salah ditebak positif.
- **FN = 11**: pasien positif yang salah ditebak negatif.
- **TP = 125**: pasien positif yang tertebak positif.

Total hanya **21 kesalahan** dari 462 data uji.




*Confusion matrix data uji*

*Classification report* memperlihatkan kinerja per kelas. Kelas 0 (negatif) mendapat presisi 0,97 dan recall 0,97, sedangkan kelas 1 (positif) presisi 0,93 dan recall 0,92. Kinerja kelas positif sedikit lebih rendah karena jumlah datanya lebih sedikit (kelas tidak seimbang), namun recall 0,92 berarti model tetap berhasil mengenali sebagian besar pasien positif. Dalam konteks diagnosis, recall kelas positif penting karena pasien positif yang lolos (FN) lebih berisiko daripada salah duga positif (FP).

## 9. Kesimpulan

Program deep learning pada modul berhasil dilengkapi dan dijalankan. Jaringan saraf tiruan dengan dua lapisan tersembunyi (16 dan 8 neuron) mencapai **akurasi 95,45%** pada data uji, dengan hanya 21 kesalahan dari 462 data. Kurva training dan validasi stabil; hanya terlihat overfitting ringan pada loss validasi setelah sekitar epoch 25 yang tidak menurunkan akurasi. Preprocessing yang tepat, yaitu memperbaiki isian umur yang salah, encoding umur, dan standarisasi fitur, ikut berperan dalam hasil tersebut.